Implementação e Validação em Hardware do Processador MIPS Monociclo com Instruções de Desvio

1st José Dercy Vieira da Silva Filho
Engenharia da Computação (FEEC)
Universidade Federal do Amazonas (UFAM)
Manaus, Brasil
jose.filho@ufam.edu.br

2nd Vinicius Menezes
Engenharia da Computação (FEEC)
Universidade Federal do Amazonas (UFAM)
Manaus, Brasil
vinicius.menezes@ufam.edu.br

3rd Isabela Furtado
Engenharia da Computação (FEEC)
Universidade Federal do Amazonas (UFAM)
Manaus, Brasil
isabela.furtado@icomp.ufam.edu.br

I. Introdução

A implementação de uma arquitetura de computadores em linguagem de descrição de hardware (HDL) constitui uma etapa fundamental para a consolidação dos conceitos de organização e projeto lógico digital [1]. O conjunto de instruções MIPS caracteriza-se por sua regularidade ortogonal e simplicidade de decodificação, tornando-o um modelo de referência amplamente adotado tanto no meio acadêmico quanto em projetos de sistemas embarcados.

Nesta segunda etapa do projeto, o objetivo central consiste na especificação, implementação estrutural e validação funcional do caminho de dados e da unidade de controle do processador MIPS em sua versão monociclo. Em relação à primeira entrega — focada na especificação do software base e no mapeamento do algoritmo Bubble Sort —, esta fase consolida os blocos de hardware necessários para executar o fluxo completo de instruções, incluindo desvios condicionais e saltos incondicionais (jumps).

II. Ambiente de Desenvolvimento e Configuração

O desenvolvimento do processador MIPS monociclo utiliza um conjunto de ferramentas integradas para síntese lógica, simulação temporal e prototipagem física:

- Intel Quartus Prime: Ambiente principal para análise estrutural, síntese lógica, mapeamento tecnológico e atribuição de pinos (*pin assignment*) para a FPGA Intel Cyclone IV / MAX 10 [2].
- Icarus Verilog & GTKWave: Plataforma de compilação rápida e visualização de formas de onda (*waveforms*) para depuração ciclo a ciclo dos sinais de controle e barramentos de dados durante o desenvolvimento inicial.
- Placa Alvo (DE10-Lite): Plataforma de prototipagem física baseada no dispositivo MAX 10 (10M50DAF484C7G), equipada com memória externa, chaves, botões táteis e displays de 7 segmentos [2].

III. Especificação do Caminho de Dados e Instruções de Salto

A arquitetura monociclo executa exatamente uma instrução por ciclo de clock. A estrutura modular dos blocos funcionais foi implementada em Verilog HDL [3] utilizando parâmetros configuráveis (parameter) para larguras de barramento de dados e endereçamento.

A. Organização Modular dos Blocos Funcionais

O processador divide-se nos seguintes submódulos principais:

- Banco de Registradores (Register File): Contém 32 registradores de 32 bits com duas portas de leitura assíncronas e uma porta de escrita síncrona sensível à borda de subida do clock.
- Unidade Lógica e Aritmética (ALU): Bloco combinacional parametrizado capaz de realizar operações lógicas (AND, OR), aritméticas (ADD, SUB) e comparações (SLT).
- Unidade de Controle: Decodifica o campo opcode e gera os sinais de controle essenciais (RegDst, Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc, RegWrite, Jump).

B. Implementação das Instruções de Desvio (Jumps)

Para suportar laços aninhados e chamadas de funções identificadas na primeira entrega, foram incorporadas as seguintes instruções de controle de fluxo:

- j (Jump incondicional tipo-J): O endereço alvo de 32 bits é formado concatenando os 4 bits mais significativos do endereço atual do Program Counter

(PC+4[31:28]), os 26 bits do campo imediato deslocados de 2 posições para a esquerda (instr[25:0] « 2).

- jal (Jump and Link): Além do salto incondicional, preserva o endereço de retorno PC+4 no registrador $ra ($31).
- jr (Jump Register tipo-R): Permite o retorno de funções carregando o Program Counter diretamente a partir do conteúdo do registrador especificado (PC <= Reg[rs]).

IV. Validação Funcional e Testbenches

Em conformidade com as diretrizes do projeto, cada componente do processador foi submetido a baterias de testes unitários através de módulos de *testbench* dedicados.

A. Testes Unitários dos Componentes

- tb_alu.v: Valida as operações aritméticas e lógicas, além do sinal de Zero Flag com vetores de teste cobrindo casos limites (como valores negativos e subtração com resultado nulo).
- tb_reg_file.v: Testa a escrita síncrona e a leitura assíncrona simultânea em registradores distintos, assegurando que o registrador $zero permaneça imutável com valor fixo 0.
- tb_control_unit.v: Atesta a decodificação exata de cada combinação de opcode e funct, verificando a consistência dos sinais de multiplexação.
- tb_instr_mem.v: Confere a leitura correta das instruções a partir de endereços alinhados por palavra de 32 bits.

B. Integração Top-Level e Simulação Geral

O módulo de topo (tb_mips_monociclo_top.v) carrega o código de máquina gerado a partir do algoritmo de ordenação (sort_swap.hex / machine_code.bin). A simulação confirma a evolução sequencial do Program Counter, os desvios condicionais (beq, bne) e os saltos incondicionais até o encerramento do programa.

V. Considerações Finais e Próximos Passos

A implementação do MIPS monociclo nesta segunda entrega permitiu consolidar a arquitetura completa do caminho de dados e unidade de controle com suporte às instruções de salto (jumps). A validação estruturada por meio de testbenches modulares assegurou a corretude comportamental prévia à síntese física no software Quartus Prime.

A apresentação detalhada do projeto, demonstração da simulação e análise das formas de onda encontram-se documentadas no vídeo de entrega disponibilizado na plataforma YouTube, cujo link e slides associados constam no repositório do projeto.

Como próximos passos para a terceira entrega, o caminho de dados será segmentado nos cinco estágios clássicos de *pipeline* (IF, ID, EX, MEM, WB), introduzindo os registradores de desacoplamento temporal e as unidades de detecção de *hazards* e encaminhamento (*forwarding*).

References

[1] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware / Software Interface, 4th ed. Academic Press, 2012.

[2] Terasic Technologies, DE10-Lite User Manual, 1st ed., Terasic Inc., 2017. [Online]. Available: https://www.terasic.com.tw/

[3] IEEE Standard for Verilog Hardware Description Language, Institute of Electrical and Electronics Engineers Std. IEEE Std 1364-2005, 2006. [Online]. Available: https://standards.ieee.org/